

# Trader System Manual

Alpaca Paper Trading Bot

LSTM + LightGBM Ensemble with 12-Layer Risk Management

Platform: NVIDIA Jetson Orin Nano Super (ARM64, 8GB)

JetPack 6.2.1 | PyTorch 2.8.0 | CUDA 12.6 | cuDNN 9.3

Broker: Alpaca Markets (Paper Trading, \$100k equity)

LLM: Google Gemini (tiered routing: Pro/Flash/Lite)

Generated: 2026-03-29

# Table of Contents

1. System Overview
2. Prediction Engine -- LSTM + LightGBM
3. Feature Engineering
4. Training Pipeline & Optuna Search
5. Trading Algorithm -- Main Loop
6. Entry Gates (Buy Logic)
7. Exit Logic (Sell, Short, Cover)
8. Position Sizing -- 12-Layer Risk Stack
9. Stop-Loss & Risk Controls
10. Horizon-Adaptive Scaling
11. LLM Integration
12. Sentiment Analysis
13. GARCH Volatility Forecasting
14. Macro Regime Detection
15. Correlation-Aware Sizing
16. HMM Regime Detection
17. Data Pipeline
18. Research Foundations

# Chapter 1

## System Overview

---

### 1.1 Purpose

This system is an automated paper trading bot for stocks and cryptocurrencies. It uses an LSTM neural network with multi-head self-attention, ensembled with LightGBM gradient-boosted trees, to predict forward returns on hourly bars. Positions are sized through a 12-layer risk management stack drawing on portfolio theory, volatility modeling, behavioral economics, and regime detection. The system trades autonomously on Alpaca's paper trading platform with \$100k equity.

The bot runs two independent trading loops: CryptoLoop (24/7, all crypto pairs) and StockLoop (market hours only, top-N by signal strength). Both inherit from a shared BaseTradingLoop via the Template Method pattern, ensuring identical risk management logic across asset classes.

### 1.2 Hardware Platform

The system runs on an NVIDIA Jetson Orin Nano Super, chosen for its GPU capability at low power consumption (15-25W). Key specifications:

- CPU: 6-core ARMv8 Cortex-A78AE @ 1728MHz
- GPU: Ampere SM\_87 (compute capability 8.7), 8 SMs, 1024 CUDA cores
- Memory: 8GB LPDDR5 unified (shared CPU/GPU)
- Storage: 465GB NVMe (primary, 381GB free), 117GB SD card (secondary)
- OS: Ubuntu 22.04, JetPack 6.2.1, Linux 5.15.148-tegra
- Power mode: MAXN\_SUPER (maximum performance)

The unified memory architecture means GPU training and CPU inference compete for the same 8GB. This drives several design decisions: inference always runs on CPU to preserve GPU for training, a GPU lock prevents concurrent GPU access, training caps CUDA allocation at 40% (~3GB), and OOM retry logic halves batch size on failure.

### 1.3 Software Stack

- Jetson env (Python 3.10): PyTorch 2.8.0, CUDA 12.6, cuDNN 9.3, Numba 0.63.1, LightGBM, Optuna, arch (GARCH), hmmlearn, alpaca-trade-api 3.2.0, fredapi
- Base env (Python 3.12): PySide6 for GUI dashboard only (PySide6 not available for ARM64 Python 3.10)

PyTorch 2.9.1 exists but is BROKEN (missing libcudss.so.0) -- do NOT upgrade. The LD\_LIBRARY\_PATH must include nvidia/cusparselt/lib for libcusparseLt.so.0.

### 1.4 Broker Integration

The system connects to Alpaca Markets paper trading via their REST API. Account equity is \$100k. API credentials (ALPACA\_API\_KEY, ALPACA\_API\_SECRET, ALPACA\_BASE\_URL) are loaded from .env via python-dotenv. The system supports both stock and crypto trading through Alpaca's unified API, with bracket orders for stocks and limit orders for crypto.

### 1.5 LLM Integration

Google Gemini provides supplementary analysis via a tiered routing system. The system auto-detects free vs paid tier from API rate limit headers and routes requests to the appropriate model (Pro for high-value analysis, Flash for mid-range, Lite when daily budget is nearly exhausted). Daily budget cap: \$1.00. The LLM serves as a supplementary

gate in the trading pipeline, not the primary signal.

# Chapter 2

## Prediction Engine -- LSTM + LightGBM

### 2.1 RegressionLSTM Architecture (model\_v2.py)

The neural network predicts continuous return percentages (regression, not classification). The architecture combines LSTM temporal modeling with multi-head self-attention for adaptive timestep weighting:

```
Input: (batch, seq_len, input_dim)    # e.g., (32, 20, 22)

-> LSTM(input_dim, hidden_dim, num_layers, dropout)
  - Unidirectional (causal -- no future leakage)
  - Dropout between layers if num_layers > 1
  - hidden_dim: 64 to 384 (tuned by Optuna)
  - num_layers: 1 or 2 (3+ overfits on this data)

-> MultiHeadAttention(hidden_dim, n_heads, dropout, batch_first=True)
  - Self-attention: Q = K = V = lstm_output
  - n_heads: 2 to 4 (must evenly divide hidden_dim)
  - Learns which timesteps in the sequence are most informative
  - Allows the model to attend to distant bars (e.g., a spike 15 bars ago)

-> Residual Connection + LayerNorm
  - output = LayerNorm(attn_out + lstm_out)
  - Stabilizes training, prevents attention from degrading LSTM signal
  - Residual ensures LSTM information is preserved even if attention
    learns nothing useful

-> Mean Pooling (dim=1)
  - Average over the sequence dimension
  - Collapses (batch, seq_len, hidden_dim) -> (batch, hidden_dim)

-> FC Head
  - Linear(hidden_dim, 64) -> ReLU -> Dropout -> Linear(64, 1)
  - Final scalar output: predicted return percentage

Output: predicted_return (%)
```

### 2.2 Hyperparameter Ranges

All architecture hyperparameters are tuned via Optuna's TPE sampler:

| Parameter  | Range                  | Notes                                             |
|------------|------------------------|---------------------------------------------------|
| hidden_dim | [64, 384] step 32      | LSTM hidden state size. Adaptive boundaries.      |
| num_layers | [1, 2]                 | LSTM depth. 3+ causes overfitting on hourly data. |
| n_heads    | [2, 4]                 | Attention heads. Must evenly divide hidden_dim.   |
| dropout    | [0.10, 0.40] step 0.05 | Applied in LSTM, attention, and FC head.          |
| seq_len    | [8, 40] step 2         | Input window length in bars. Adaptive boundaries. |

### 2.3 LightGBM Stacking (model\_lgb.py)

LightGBM is trained on flattened LSTM input sequences. The (seq\_len x n\_features) tensor is reshaped into a 1D vector with lag-named columns (e.g., RSI, RSI\_lag1, RSI\_lag2, ...). This tree-based model captures non-linear feature interactions and threshold effects that the LSTM may miss. Literature shows 10-15% Sharpe improvement from LSTM+tree stacking.

```
LightGBM hyperparameters:
num_leaves=63, learning_rate=0.05, feature_fraction=0.7
bagging_fraction=0.8, bagging_freq=5
```

500 boosting rounds, early stopping after 20 rounds without improvement

Training data: Same walk-forward folds as LSTM (no data leakage)

Features: Flattened sequence (seq\_len \* n\_features columns)

## 2.4 Ensemble Weighting

The final prediction is a weighted average of both models:

```
predicted_return = 0.6 * lstm_pred + 0.4 * lgb_pred
```

Rationale:

- LSTM captures temporal dynamics and sequential patterns
- LightGBM captures static feature interactions and thresholds
- 60/40 weighting reflects LSTM's stronger performance on temporal data
- If LGB model is unavailable: falls back to LSTM-only (graceful degradation)

## 2.5 Loss Function

Training uses Huber loss with return-weighted samples. Huber loss is less sensitive to outliers than MSE (important in financial data with fat tails), while still providing gradient signal for small errors unlike MAE:

```
raw_loss = HuberLoss(delta=huber_delta, reduction='none')
weights = clamp(|actual_return| + 1.0, max=50.0)
loss = (raw_loss * weights).mean()
```

huber\_delta: tuned by Optuna in range [0.5, 2.0]

Effect of return-weighting:

- A 5% actual return gets weight 6.0 (5.0 + 1.0)
- A 0.1% actual return gets weight 1.1 (0.1 + 1.0)
- > Model learns to predict large moves accurately (where profit is)
- > Small noise near zero is de-emphasized

## 2.6 Inference Pipeline (predict\_now.py)

Inference runs on CPU (even when GPU is available) to preserve GPU memory for training. JIT tracing provides 10-20% speedup on CPU:

```
For each symbol (parallel, 5 threads via ThreadPoolExecutor):
1. Fetch recent bars (120 crypto / 200 stock hourly)
   - Primary: Alpaca API
   - Fallback: yfinance
2. Compute technical features (indicators.py, 40+ indicators)
3. Inject Daily_Sentiment (if model was trained with it)
   - Lazy import sentiment_history.get_live_daily_sentiment()
   - Fallback to 0.0 if unavailable
4. Scale features with RobustScaler (fitted on training data)
5. Extract last seq_len bars as input tensor
6. LSTM inference (JIT-traced model, CPU, <10ms per symbol)
7. LightGBM prediction on flattened features (if model available)
8. Weighted ensemble -> predicted_return %
9. Optional: return snapshot dict of latest indicator values
```

# Chapter 3

## Feature Engineering

---

### 3.1 Implementation Stack

Features are computed through a three-tier implementation stack, selected automatically at import time for maximum performance on ARM64:

- Tier 1 -- C extension (indicators\_c.so): ARM64 NEON-optimized native binary compiled via Numba AOT. Fastest path, used when available.
- Tier 2 -- Numba JIT (@njit): Compiled on first call, 3-10x speedup over pure Python. Automatic fallback if C extension unavailable.
- Tier 3 -- Pure NumPy/Pandas: Always-available fallback. No compilation required.

### 3.2 Momentum Indicators

- RSI(14): Relative Strength Index. Bounded [0, 100]. Measures speed and change of price movements. >70 overbought, <30 oversold.
- MACD(12,26,9): Moving Average Convergence Divergence. Three components: MACD line (fast EMA - slow EMA), signal line (9-period EMA of MACD), histogram (MACD - signal). Captures momentum shifts and trend direction.
- ROC(12): Rate of Change over 12 periods. Simple percentage return. Stationary by construction.

### 3.3 Volatility Indicators

- ATR(14): Average True Range. Measures volatility as the average of true ranges (max of high-low, |high-prev\_close|, |low-prev\_close|). Used for stop-loss placement and position sizing.
- Bollinger Bands(20,2): Upper/lower bands at +/-2 standard deviations from 20-period SMA. Features: BBP (percent B, position within bands, bounded 0-1) and BBB (bandwidth, measures volatility expansion/contraction).

### 3.4 Oscillators

- Stochastic(14,3,3): %K and %D lines. Bounded [0, 100]. Compares closing price to high-low range over 14 periods. %K is raw, %D is 3-period SMA of %K. Overbought >80, oversold <20.

### 3.5 Volume Indicators

- OBV: On Balance Volume. Cumulative volume where up days add volume, down days subtract. Confirms price trends (rising OBV + rising price = strong trend).
- Volume\_Ratio: Current volume / rolling average volume. Stationary by construction. Spikes indicate unusual activity (earnings, news).

### 3.6 Trend Indicators

- SMA\_20, SMA\_100: Simple Moving Averages. Used to compute ratios.
- Price\_SMA20\_Ratio, Price\_SMA100\_Ratio: Price divided by SMA. Stationary, centered around 1.0. Values >1 = above trend, <1 = below trend.

### 3.7 Advanced Features

## Hurst Exponent (R/S Analysis)

Measures persistence in the price series using Rescaled Range analysis with a rolling window of 100 bars. This is a key regime indicator that determines whether momentum strategies or mean-reversion strategies are appropriate:

```
H > 0.5: Trending (persistent momentum -- favor momentum signals)
H ~ 0.5: Random walk (low predictability -- reduce confidence)
H < 0.5: Mean-reverting (anti-persistent -- momentum signals unreliable)
```

```
Implementation: _hurst_rs(arr, window=100)
1. For each rolling window: compute mean, cumulative deviation
2. R = max(cumdev) - min(cumdev), S = std(returns)
3. H = log(R/S) / log(window)
Bounded [0, 1], stationary
```

## Calendar Features

Cyclical time encoding captures intraday, weekly, and seasonal patterns without creating artificial discontinuities (e.g., hour 23 is close to hour 0):

```
Hour_sin = sin(2 * pi * hour / 24)    Hour_cos = cos(2 * pi * hour / 24)
Day_sin  = sin(2 * pi * dayofweek / 7) Day_cos  = cos(2 * pi * dayofweek / 7)
Month_sin = sin(2 * pi * month / 12)   Month_cos = cos(2 * pi * month / 12)
```

```
Turn_of_Month: binary flag for last 2 and first 2 trading days of month.
Captures the well-documented Turn-of-Month effect (Ariel 1987, Lakonishok
& Smidt 1988) where returns are disproportionately concentrated around
month boundaries due to institutional cash flows.
```

## 3.8 Cross-Asset Features

Crypto-only (BTC as market leader):

```
BTC_Return_1h -- Bitcoin hourly return (altcoins lag BTC by 1-4 hours)
BTC_SMA_Ratio -- BTC price vs SMA (BTC trend state)
BTC_RSI       -- BTC momentum (altcoin beta to BTC is ~1.3)
```

Stock-only:

```
RS_vs_SPY     -- Relative strength vs S&P 500 (sector rotation signal)
Price_VWAP_Ratio -- Intraday strength (price / VWAP, >1 = strong)
Gap_Pct       -- Overnight gap: (open - prev_close) / prev_close
ATR_Pct       -- Normalized volatility: ATR / close * 100
```

## 3.9 Feature Presets

Four presets control which features the model trains on. The preset is selected in `indicator_config.json` and can be changed via the GUI Settings tab:

| Preset     | Count | Description                                                       |
|------------|-------|-------------------------------------------------------------------|
| minimal    | 18    | Core oscillators + returns only. Fast training, lower accuracy.   |
| stationary | 20    | All bounded/stationary features. No trending data (SMA levels).   |
| standard   | 30+   | Stationary + trend ratios + volume. Recommended starting point.   |
| full       | all   | Every computed feature including raw levels. Risk of overfitting. |

The 'stationary' preset is recommended for regression models. All features are bounded or stationary by construction, preventing distribution drift that degrades model accuracy over time.

## Chapter 4

### Training Pipeline & Optuna Search

#### 4.1 Pipeline Orchestrator (run\_pipeline.py)

Training is managed by a multi-phase pipeline orchestrator that handles data harvesting, hyperparameter search, model selection, and bot lifecycle:

```
Phase A: Initial Training
  1. Harvest: Alpaca -> yfinance -> CryptoCompare (3-source fallback)
  2. Optuna search: 200 trials per model (crypto + stock)
  3. Save best model: model_v2.pth, config_v2.pkl, scaler_v2.pkl

Phase B: Trading (continuous)
  - Start crypto + stock bots as subprocesses
  - Monitor & auto-restart on crash
  - Bots hot-reload models when .pth file mtime changes

Phase C: Weekly Retrain (Saturday 2 AM or manual via GUI)
  - Stop bots (free GPU memory)
  - Adaptive mode: refine (70 trials) or explore (120 trials)
  - Incremental data harvest (only new bars since last run)
  - Search with potentially expanded hyperparameter space
  - Restart bots with new models (if score improved)
```

#### 4.2 Optuna Study Configuration

```
study = optuna.create_study(
    storage='sqlite:///{{prefix}}v2_study.db', # Resumable across runs
    direction='maximize', # Maximize risk-adjusted Sharpe
    sampler=TPESampler(n_startup_trials=60), # Bayesian optimization
    pruner=MedianPruner(n_startup_trials=60, n_warmup_steps=12)
)
```

TPE (Tree-structured Parzen Estimator) is a Bayesian optimization algorithm that models the relationship between hyperparameters and performance using kernel density estimation. After 60 random startup trials to build the density model, it focuses exploration on promising regions of the search space. MedianPruner terminates unpromising trials early -- those performing below the running median of completed trials -- after 12 warmup epochs, saving significant GPU time.

#### 4.3 Hyperparameter Search Space

| Parameter      | Range                  | Notes                                              |
|----------------|------------------------|----------------------------------------------------|
| forwardBars    | [12,18,24,32,48]       | Prediction horizon (bars ahead). Expandable to 96. |
| seqLen         | [8, 40] step 2         | LSTM input window. Adaptive low/high boundaries.   |
| hiddenDim      | [64, 384] step 32      | LSTM hidden size. Adaptive boundaries.             |
| numLayers      | [1, 2]                 | LSTM depth. 3+ overfits on hourly data.            |
| nHeads         | [2, 4]                 | Attention heads. Must divide hiddenDim evenly.     |
| dropout        | [0.10, 0.40] step 0.05 | Regularization strength.                           |
| learningRate   | [5e-4, 3e-3] log       | Adam optimizer LR. Log-uniform sampling.           |
| batchSize      | [512, 1024, 2048]      | Larger = faster but OOM risk on Jetson 8GB.        |
| weightDecay    | [1e-5, 5e-4] log       | L2 regularization for Adam.                        |
| huberDelta     | [0.5, 2.0] step 0.1    | Huber loss robustness threshold.                   |
| tradeThreshold | [0.05, 1.0] step 0.01  | Minimum predicted return to trade.                 |
| scheduler      | [cosine, plateau]      | LR annealing strategy.                             |

## 4.4 Walk-Forward Cross-Validation (3-Fold)

Expanding window strategy with embargo gap to prevent look-ahead bias. The embargo gap equals `seq_len` bars, ensuring no overlapping sequences between training and validation:

```
Fold 0: train on first 60% | embargo (seq_len bars) | val on next 14%
Fold 1: train on first 73% | embargo (seq_len bars) | val on next 14%
Fold 2: train on first 86% | embargo (seq_len bars) | val on next 14%
```

Key properties:

- Expanding window: each fold sees MORE training data (mimics production)
- Embargo gap = `seq_len * EMBARGO_MULTIPLIER` (default 1x) bars  
Prevents target leakage from overlapping sequences
- Per-ticker chronological splits: within each ticker, data is ordered by timestamp. No future data leaks to the training set.
- Validation set always follows training set in time (temporal ordering)

## 4.5 Training Loop Internals

For each fold (up to `MAX_EPOCHS=60`):

- Loss: Return-weighted Huber loss (see Chapter 2)
- Precision: FP16 mixed precision (`torch.amp.GradScaler` on CUDA)
- Gradient clipping: `max_norm=1.0` (prevents gradient explosion)
- Early stopping: 10 epochs patience on validation loss
- OOM retry: halves batch size (min 128), clears CUDA cache, retries
- Trial timeout: 900s per trial (15 minutes max)

GPU memory management:

- `torch.cuda.set_per_process_memory_fraction(0.40) = ~3.2GB` on 8GB Jetson
- This leaves headroom for system processes and prevents OOM kills

## 4.6 Sharpe Ratio Objective

The optimization objective is a risk-adjusted Sharpe ratio computed from simulated trading on validation data:

Per-fold Sharpe calculation:

```
signal = +1 if pred > threshold, -1 if pred < -threshold, 0 otherwise
trade_return = signal * actual_return - 5 bps (transaction cost per trade)
annualized_sharpe = (mean(trade_returns) / std(trade_returns)) *
                    sqrt(annualized_trade_frequency)
```

Fold aggregation (risk-adjusted):

```
score = mean(fold_sharpes) - 0.5 * std(fold_sharpes)
-> Penalizes inconsistency across folds
-> A model with Sharpe [2.0, 0.5, 1.0] scores lower than [1.2, 1.1, 1.0]
```

Regime-aware penalty:

- Regime detection: rolling 50-bar return > 2% = bull, < -2% = bear, else sideways
- If ANY regime (bull/bear/sideways) has Sharpe < -0.5: score \*= 0.7
- > 30% score reduction ensures model works in ALL market conditions
- > Prevents models that only profit in bull markets

## 4.7 Adaptive Search Modes

The search space and trial count adapt automatically based on previous results:

Modes and trial counts:

```
INITIAL: 200 trials (first run, widest exploration)
REFINE:  70 trials (exploit local optimum, narrow ranges around best)
EXPLORE: 120 trials (wider search after edge detection or stagnation)
```

Transition rules:

- > EXPLORE: if ANY hyperparameter hits search space boundary  
OR 3+ cycles without >5% score improvement
- > REFINE: after explore completes (toggle back to exploitation)

```
State persisted to adaptive_state_{asset_type}.json:
best_score, best_params, search_space, mode,
cycles_without_improvement, expansion_history
```

## 4.8 Edge Detection & Space Expansion

When the best hyperparameter value is at or near a search space boundary, the true optimum may lie outside the current range. The system detects this and expands:

```
Detection: check if best value within 10% of boundary
Example: seq_len range [8, 40], best=8
(8-8)/(40-8) = 0.0 <= 0.10 -> LOW EDGE DETECTED

Expansion pools (next value added to boundary):
forward_bars: low=[8], high=[64, 96]
seq_len: low=[4], high=[48]
hidden_dim: low=[32], high=[512]

Hard limits prevent unbounded growth:
seq_len: [4, 64], hidden_dim: [32, 512], etc.

If categorical parameter changes: Optuna study DB is DELETED
(incompatible with new search space)
```

## 4.9 Model Protection

A new model is only saved to disk if its score exceeds the `best_score` recorded in `adaptive_state`. This prevents regression -- a bad retrain cycle cannot overwrite a good model. Files saved on improvement: `model_v2.pth` (weights), `config_v2.pkl` (hyperparams), `scaler_v2.pkl` (RobustScaler fitted on training data), `feature_cols_v2.pkl` (column names ensuring train/inference feature alignment).

Current best: stock model score=11.009, forward\_bars=96 (96-hour prediction horizon).

## 4.10 Forward Bars

The `forward_bars` parameter defines the prediction horizon -- how many bars ahead the model predicts returns. The default search set is [12, 18, 24, 32, 48] hours, expandable to 96 via edge detection. Longer horizons give the model more signal (larger moves to predict) but require wider stops and longer cooldowns. All risk parameters scale adaptively with `forward_bars` (see Chapter 10).

# Chapter 5

## Trading Algorithm -- Main Loop

### 5.1 Loop Cycle (every 30 seconds)

The main trading loop runs a fixed sequence of operations every 30 seconds. Both CryptoLoop and StockLoop inherit this skeleton from BaseTradingLoop:

```
run():
    while True:
        check_market_hours()           # Crypto: always True; Stocks: 9:30-4:00 ET
        _circuit_breaker_check()       # 5% daily drawdown -> flatten + 1h sleep
        flatten_before_close()         # Stocks: sell all at 3:50 PM ET
        _hot_reload_check()            # Reload model if .pth mtime changed

        [every 10 cycles = ~5 minutes]:
            _update_macro_regime()      # VIX, STLFSI2, CAPE, stablecoins
            _update_equity()            # Track peak equity for drawdown
            _update_correlations()      # 30-day rolling Pearson matrix
            log GPU temperature         # Thermal monitoring

        _manage_stops()                # Hard stop, trailing stop, take-profit
        preds = _get_predictions()      # Parallel LSTM+LGB inference (5 threads)
        _run_llm_analysis(preds)        # LLM scoring (throttled to 1 hour)
        _execute_sells(preds)          # Sell if bearish + LLM confirms
        _execute_llm_veto_sells()       # Sell if LLM score < 0.15 (catastrophic)
        _execute_covers(preds)         # Cover shorts if bullish or stopped
        _execute_shorts(preds)         # Short if strongly bearish
        _execute_buys(preds)           # Buy with full 12-layer risk stack
        _sleep(30s + jitter)           # Thermal throttle if GPU > 75C
```

### 5.2 Horizon-Adaptive Parameters

All trading parameters scale with the prediction horizon (forwardBars) using  $\sqrt{\text{forwardBars}/5}$  as the scaling factor. This ensures that stops, cooldowns, and thresholds are appropriately calibrated whether the model predicts 12 hours or 96 hours ahead. See Chapter 10 for full details.

### 5.3 Min Return Floor

Before any trade, the predicted return must exceed a minimum floor derived from the risk-free rate. This ensures the model predicts enough upside to justify the risk:

```
min_return_floor = 10Y_Treasury_yield * (forwardBars / 252) * 2.0
    -> Must predict > 2x the risk-free rate for the holding period

min_short_floor = 10Y_Treasury_yield * (forwardBars / 252) * 5.0
    -> Must predict > 5x risk-free rate to short (higher bar for shorts)

Example with 4.5% 10Y yield and forwardBars=96:
    min_return_floor = 0.045 * (96/252) * 2.0 = 3.43%
    min_short_floor  = 0.045 * (96/252) * 5.0 = 8.57%
```

### 5.4 Hot-Reload

Every cycle checks if the model .pth file modification time has changed. If so, the model, scaler, feature\_cols, and config are reloaded from disk. This allows the weekly retrain pipeline to update models while bots are running without any restart. The reload is atomic -- all four files are loaded together or none are updated.

# Chapter 6

## Entry Gates (Buy Logic)

Before any buy order is placed, the candidate symbol must pass ALL 11 sequential gates. If any gate rejects, the symbol is skipped for this cycle. The gates are ordered from cheapest to most expensive computation:

### Gate 1: Position Check

The symbol must not already be held long, and must not have an active short position. Prevents doubling down and ensures clean position management.

### Gate 2: Cooldown

The symbol must not have been traded within the cooldown window. Cooldown scales linearly with the prediction horizon to prevent whipsaw re-entry on longer timeframes:

```
cooldown = base_cooldown * (forwardBars / base_forwardBars)
```

Examples:

```
forwardBars=5:  cooldown = 60 minutes (base)
forwardBars=24:  cooldown = ~5 hours
forwardBars=96:  cooldown = ~19 hours
```

### Gate 3: Hard-Stop Lockout

If the symbol hit a hard stop-loss recently, it is locked out for 24-48 hours. This prevents 'revenge trading' -- the behavioral tendency to immediately re-enter after a loss, often with impaired judgment. Lockout is persisted to disk so it survives bot restarts.

### Gate 4: Prediction Exceeds Min Return Floor

The predicted return must exceed the min\_return\_floor (2x risk-free rate for the holding period). This ensures the model sees enough upside to justify taking risk. A prediction of +0.5% is meaningless if the risk-free rate earns +0.4% over the same period.

### Gate 5: Quote Available + Spread Check

A valid bid/ask quote must be available from the broker. The predicted return must exceed 2x the round-trip spread cost (spread-to-reward ratio). This ensures the edge exceeds market friction:

```
Required: pred_return > 2 * spread_pct (round-trip)
```

```
Example: spread = 0.05% -> pred must be > 0.10%
If pred = 0.08%, the edge is consumed by spread costs
```

### Gate 6: Winner's Curse Filter

Inspired by Milgrom & Wilson's auction theory (Nobel 2020). When price has extended beyond SMA\_20 + 2\*ATR, the buyer is likely overpaying for momentum that's about to reverse. In this condition, the prediction threshold is raised to 1.5x to require much stronger conviction before entering:

```
if price > SMA_20 + 2 * ATR:
    effective_threshold = threshold * 1.5
    -> Requires 50% stronger signal to enter late-stage rallies
```

## Gate 7: Correlation Check

The average absolute correlation between the candidate and all currently held positions must be below 0.7. If above, entry is blocked entirely to prevent portfolio concentration. Based on Markowitz portfolio theory. See Chapter 15 for full details.

## Gate 8: Macro Regime

Macro conditions can block entry entirely:

- VIX > 35: Halt ALL stock entries (crisis mode). Crypto unaffected.
- VIX 25-35: Block non-safe-haven stocks (only TLT, GLD, etc. allowed).
- Stablecoin depeg > 2%: Halt ALL crypto entries.

## Gate 9: Position Sizing (12-Layer Stack)

The 12-layer risk stack (Chapter 8) computes the position size. If the final sized amount is below the minimum notional, entry is skipped. The sizing stack can effectively veto a trade by reducing size to zero.

## Gate 10: Sentiment Gate

News-based sentiment scoring applies a multiplier to position size. Catastrophic news (score < -0.5) reduces size by 85%. Strong bullish news (score > 0.4) increases size by 35%. The multiplier range is 0.15x to 1.35x. See Chapter 12 for details.

## Gate 11: LLM Gate

The LLM score must exceed BUY\_LLM\_MIN (approximately 0.60, adjusted by horizon). If the score is available, it also applies a sizing multiplier of  $(0.5 + \text{score})$ , ranging from 0.65x to 1.5x. A score below 0.15 is a hard veto that prevents entry regardless of all other signals.

# Chapter 7

## Exit Logic (Sell, Short, Cover)

### 7.1 Sell Hysteresis

Selling requires BOTH a bearish prediction AND bearish LLM confirmation. This dual-confirmation prevents churn from noisy signals:

```
Sell requires BOTH:
  pred < -(threshold * SELL_PRED_MULT)    # Model is bearish
  llm_score < SELL_LLM_MAX                # LLM confirms bearish view
```

```
Dead zone prevents oscillation:
  Buy zone: LLM > 0.60 (approximately, adjusted by horizon)
  Dead zone: LLM 0.40 to 0.60 (no action -- hold current position)
  Sell zone: LLM < 0.40 (approximately, adjusted by horizon)
```

```
The gap between buy (0.60) and sell (0.40) thresholds ensures that
minor LLM score fluctuations do not trigger buy-sell-buy-sell churn.
```

### 7.2 Emergency Sell

If the prediction is extremely bearish (beyond the short threshold, i.e., 5x risk-free rate), an emergency sell is triggered regardless of LLM score. This bypasses the normal hysteresis to protect against sharp drawdowns when the model has very high conviction in a downturn.

### 7.3 LLM Veto Sell

Every cycle, all held positions are checked against their LLM scores. If any position has LLM score < 0.15 (catastrophic zone), it is immediately liquidated. This overrides all other logic -- the LLM has identified a severe risk (fraud, hack, regulatory action) that the quantitative model cannot see in price data alone.

### 7.4 Short Entry

Short selling has a higher bar than long entry due to unlimited downside risk:

```
Short entry requires ALL:
  pred < -min_short_floor                # 5x risk-free rate (very bearish)
  llm_score < 0.30                       # LLM strongly bearish
  Not a crypto symbol                    # No crypto shorting
  Not a leveraged ETF                    # No leveraged shorting
  Position size = half normal            # Half-sized for risk control
```

```
Short positions use the same stop-loss and take-profit logic as longs,
but inverted (stop above entry, take-profit below entry).
```

### 7.5 Short Cover

Short positions are covered (closed) when any of these conditions are met:

- Prediction flips bullish (pred > threshold)
- LLM score rises above 0.65 (sentiment reversal)
- Stop-loss triggered (price rises above entry + stop\_dist)
- Daily flatten for stocks at 3:50 PM ET

## 7.6 Daily Flatten (Stocks Only)

At 3:50 PM ET, all stock positions (both longs and shorts) are closed to avoid overnight gap risk. This protects against earnings announcements, geopolitical shocks, and overnight gap-downs that the intraday model cannot predict. Crypto never flattens since markets never close.

# Chapter 8

## Position Sizing -- 12-Layer Risk Stack

Position sizing passes through 12 multiplicative layers. Each layer can independently reduce (or modestly increase) the base notional. The design philosophy: multiple weak signals compound into strong risk management. No single layer dominates, and the stack is resilient to any individual layer failing.

### Step 1: Kelly Criterion (Half-Kelly)

The Kelly Criterion (Kelly, 1956) computes the optimal fraction of capital to risk on each trade, maximizing long-term geometric growth rate. We use half-Kelly because: (1) it yields 75% of full-Kelly's return with only 50% of the variance, (2) full-Kelly drawdowns commonly exceed 50%, and (3) estimation error in win rate and payoff ratios makes full-Kelly dangerously aggressive:

```
kelly_f = (win_rate * avg_win/avg_loss - (1 - win_rate)) / (avg_win/avg_loss)
half_kelly = kelly_f / 2, clamped to [0.05, 0.25]
position_size = half_kelly * equity
```

Data source: last 200 trades in trade\_memory.json  
Minimum: 50 trades before Kelly activates (else fixed NOTIONAL)

### Step 2: VIX Scaling

```
VIX > 35: kelly_scale = 0.3 (crisis -- near halt)
VIX 25-35: kelly_scale = 0.5 (defensive)
VIX 15-25: kelly_scale = 0.7 (caution)
VIX < 15: kelly_scale = 1.0 (normal -- full size)
```

### Step 3: Drawdown-Based Reduction

Tracks peak equity and reduces sizing when underwater. This prevents compounding losses during losing streaks:

```
drawdown = (peak_equity - current_equity) / peak_equity
dd >= 20%: base *= 0.25 (75% reduction -- near halt)
dd >= 15%: base *= 0.50 (50% reduction)
dd >= 10%: base *= 0.75 (25% reduction)
dd < 10%: no change
```

### Step 4: Confidence Scaling

Scales position size proportionally to prediction strength. The denominator is the min\_return\_floor (not the noise threshold), ensuring confidence reflects excess return over the risk-free hurdle:

```
confidence = clamp(pred_return / min_return_floor, 0.5, 2.0)
sized = base * confidence
```

Effect: Stronger predictions get bigger positions.  
pred = 2x floor -> confidence = 2.0 -> double size  
pred = 0.5x floor -> confidence = 0.5 -> half size

### Step 5: GARCH(1,1) Volatility Targeting

Based on Robert Engle's GARCH model (Nobel Prize 2003). Each position targets the same daily volatility (2% default), regardless of the asset's actual volatility:

```
sigma = GARCH/EGARCH 1-step-ahead forecast (decimal, e.g. 0.03)
ratio = target_vol / sigma = 0.02 / 0.03 = 0.67
sized *= clamp(ratio, 0.5, 2.0)
```

Effect: High-vol assets get smaller positions, low-vol get larger  
 -> Equalizes dollar risk across portfolio (risk parity)

## Step 6: Macro Regime Multiplier

Combines VIX, STLFSI2, CAPE, and stablecoin peg data into a single sizing multiplier. See Chapter 14 for the full macro regime detection system.

## Step 7: Correlation Reduction (Markowitz)

```
avg_corr = mean(abs(corr(candidate, each_held_position)))
sizing_factor = max(0.5, 1.0 - 0.5 * avg_corr)
```

Examples:

- corr = 0.0 -> factor = 1.0x (uncorrelated, full size)
- corr = 0.35 -> factor = 0.825x (moderate reduction)
- corr = 0.7 -> REJECTED (blocked by Gate 7, never reaches here)

## Step 8: HMM Regime Scaling

3-state Gaussian HMM classifies the current market regime:

```
Bull regime:   sizing = 1.2x (lean in)
Neutral regime: sizing = 1.0x (no change)
Bear regime:   sizing = 0.3x (70% reduction, very defensive)
```

## Step 9: Ensemble Disagreement Penalty

When the macro regime (VIX-based) and HMM regime disagree on market conditions, a 20% penalty is applied. Rationale: if two independent regime detectors see different things, uncertainty is high and exposure should be reduced:

```
if macro_regime != hmm_regime:
    sized *= 0.8 (20% penalty for disagreement)
```

## Step 10: Leveraged ETF Scaling

Leveraged ETFs (e.g., TQQQ 3x, SOXL 3x) have their position size divided by the leverage factor. This ensures dollar risk is equivalent to the underlying asset. A \$1,000 position in TQQQ has the same risk as \$3,000 in QQQ.

## Step 11: Sentiment Gate Multiplier

```
News-based sentiment multiplier:
score <= -0.5: mult = 0.15 (catastrophic: hack, fraud, bankruptcy)
score <= -0.3: mult = 0.35 (heavy bearish news)
score <= -0.1: mult = 0.70 (mild caution)
score >= 0.4:  mult = 1.35 (strong positive news)
score >= 0.2:  mult = 1.20 (positive news)
else:         mult = 1.00 (neutral)
Final clamp: [0.15, 1.35]
```

## Step 12: LLM Score Multiplier

```
If LLM score available:
    sized *= (0.5 + llm_score)
```

- LLM score 0.15 -> mult = 0.65x (barely above veto)
- LLM score 0.50 -> mult = 1.00x (neutral)
- LLM score 0.70 -> mult = 1.20x (bullish confirmation)
- LLM score 1.00 -> mult = 1.50x (maximum conviction)
- Range: [0.65, 1.50]

## Complete Sizing Example

AAPL trade: pred=+3.5%, min\_return\_floor=3.43%, VIX=22, equity=\$100k

|                       |                           |                                               |
|-----------------------|---------------------------|-----------------------------------------------|
| Step 1                | (Kelly, half-kelly=0.12): | $0.12 * \$100k = \$12k$ , cap \$5k -> \$5,000 |
| Step 2                | (VIX=22, caution):        | $\$5,000 * 0.7 = \$3,500$                     |
| Step 3                | (drawdown=8%, no DD):     | \$3,500 (unchanged)                           |
| Step 4                | (confidence=3.5/3.43):    | $\$3,500 * 1.02 = \$3,570$                    |
| Step 5                | (GARCH sigma=0.018):      | $\$3,570 * (0.02/0.018) = \$3,967$ , cap 2x   |
| Step 6                | (macro, VIX=22):          | $\$3,967 * 0.8 = \$3,173$                     |
| Step 7                | (corr with MSFT=0.5):     | $\$3,173 * 0.75 = \$2,380$                    |
| Step 8                | (HMM neutral, 1.0x):      | \$2,380                                       |
| Step 9                | (macro=HMM, no penalty):  | \$2,380                                       |
| Step 10               | (not leveraged):          | \$2,380                                       |
| Step 11               | (sentiment=0.3, 1.2x):    | \$2,856                                       |
| Step 12               | (LLM=0.7, 1.2x):          | \$3,427                                       |
| Final order notional: |                           | \$3,427                                       |

# Chapter 9

## Stop-Loss & Risk Controls

### 9.1 ATR-Based Dynamic Stops

Stop distances are computed from Average True Range (ATR), a volatility measure. This means volatile assets get wider stops (avoiding whipsaw) and calm assets get tighter stops (locking in gains sooner). All stop parameters scale with the prediction horizon via  $\sqrt{\text{forward\_bars}/5}$ :

```
stop_dist = (entry_atr * ATR_STOP_MULTIPLIER) / entry_price
stop_dist = clamp(stop_dist, FLOOR_PCT, CEIL_PCT)
stop_dist *= macro_regime.stop_mult  (tighten when stressed)

Horizon scaling (forward_bars=96 example):
scale = sqrt(96/5) = 4.38
FLOOR_PCT: base 5% * 4.38 = ~22%
CEIL_PCT:  base 10% * 4.38 = ~40%
-> Wider stops for longer horizons (more time = more volatility)
```

### 9.2 Trailing Stop

The trailing stop activates after a position gains `ATR_TRAIL_ACTIVATE_PCT` (default approximately 1% of entry price, scaled by horizon). Once activated, it tracks the high-water-mark and sells if price drops below  $\text{HWM} * (1 - \text{trail\_dist})$ :

```
Every cycle, for each held position:
1. Update high_water_mark = max(hwm, current_midpoint)

2. Activation check:
If hwm >= entry * (1 + ATR_TRAIL_ACTIVATE_PCT):
    trailing_activated = True

3. Trailing stop check (only if activated):
If price <= hwm * (1 - trail_dist):
    -> SELL, record exit_reason='trailing'
    -> Locks in profit as price rises, exits on pullback
```

### 9.3 Take-Profit

Take-profit targets are set at a 3:1 risk-reward ratio relative to the stop distance, capped at `TAKE_PROFIT_CEIL_PCT`:

```
tp_price = entry_price * (1 + stop_dist * TAKE_PROFIT_RR)
tp_price = min(tp_price, entry_price * (1 + TAKE_PROFIT_CEIL_PCT))

TAKE_PROFIT_RR = 3.0 (3:1 risk-reward)
TAKE_PROFIT_CEIL_PCT scales with horizon

Example with 5% stop:
tp_price = entry * (1 + 0.05 * 3.0) = entry * 1.15 (15% gain target)
```

### 9.4 Macro Regime Tightening

When the macro regime detects stress (elevated VIX, high STLFSI2, stablecoin instability), stop distances are tightened via `regime.stop_mult`. This reduces the maximum loss per position during volatile markets:

```
stop_dist *= macro_regime.stop_mult

Normal:          stop_mult = 1.0  (no change)
STLFSI2 > 1.0:  stop_mult = 0.8  (20% tighter)
Stablecoin depeg > 0.5%: stop_mult = 0.7 (30% tighter, crypto only)
```

## 9.5 Bracket Orders (Stocks)

Stock positions use Alpaca bracket orders (server-side stops). When the parent buy order fills, child stop-loss and take-profit orders are automatically created by the broker. This provides protection even if the bot process crashes. After +1% gain, the server-side stop is upgraded to a trailing stop that adjusts dynamically.

## 9.6 Circuit Breaker

If the portfolio suffers a 5% daily drawdown, the circuit breaker triggers an emergency response:

```
Trigger: (last_equity - current_equity) / last_equity >= 5%
```

Actions:

1. Cancel all open orders immediately
2. Market-sell all positions (longs and shorts)
3. Sleep for 1 hour (no new trades)
4. Resume monitoring after sleep

Rationale: Aligns with institutional risk management practice.

Prevents cascading losses during flash crashes or black swan events.

## 9.7 Hard-Stop Lockout

After a hard stop-loss fill, the symbol is locked out for 24-48 hours. This prevents 'revenge trading' -- the behavioral tendency (documented by Kahneman) to immediately re-enter after a loss, often increasing position size to 'make it back'. Lockout timestamps are persisted to `hard_stop_lockout.json` to survive bot restarts.

## 9.8 Broker Desync Detection

Before executing a sell, the system verifies the position actually exists at the broker. This prevents a dangerous failure mode where the bot's internal state shows a long position, but the broker has already filled a stop-loss. Without this check, selling a non-existent position would accidentally open a short.

# Chapter 10

## Horizon-Adaptive Scaling

All risk parameters derive from the `forwardBars` prediction horizon. This ensures that a model predicting 96 hours ahead uses appropriately wider stops, longer cooldowns, and higher return thresholds than a model predicting 12 hours ahead. The fundamental scaling law is based on the square root of time, which is how volatility scales under the random walk assumption.

### 10.1 Core Scaling Formula

```
scale = sqrt(forwardBars / 5)

forwardBars=5:  scale = 1.0    (baseline)
forwardBars=12: scale = 1.55
forwardBars=24: scale = 2.19
forwardBars=48: scale = 3.10
forwardBars=96: scale = 4.38
```

### 10.2 Stop Distances

ATR multiplier, floor, and ceiling all scale with  $\sqrt{\text{time}}$  because price volatility grows with the square root of the holding period:

```
ATR_STOP_MULTIPLIER *= scale
FLOOR_PCT *= scale      # e.g., 5% base -> 22% at 96 bars
CEIL_PCT *= scale       # e.g., 10% base -> 44% at 96 bars
```

Rationale: A 96-hour position needs wider stops because price can naturally fluctuate more over 4 days than over 5 hours. Tight stops on long-horizon positions would cause constant stop-outs.

### 10.3 Cooldown Period

Cooldown scales linearly (not  $\sqrt{\text{time}}$ ) with horizon because it represents a minimum holding period, not a volatility measure:

```
cooldown = baseCooldown * (forwardBars / baseForwardBars)

forwardBars=5:  cooldown = 60 minutes (base)
forwardBars=12: cooldown = ~2.4 hours
forwardBars=24: cooldown = ~5 hours
forwardBars=48: cooldown = ~10 hours
forwardBars=96: cooldown = ~19 hours
```

### 10.4 LLM Hysteresis Thresholds

The gap between buy and sell LLM thresholds widens slightly with horizon to reduce churn on longer-term positions:

```
Buy LLM threshold:  0.55 to 0.60 (higher for longer horizons)
Sell LLM threshold: 0.40 to 0.45 (higher for longer horizons)
```

Dead zone: the gap between buy and sell thresholds

```
Short horizon (5 bars):  dead zone = 0.55 - 0.40 = 0.15
Long horizon (96 bars):  dead zone = 0.60 - 0.45 = 0.15
```

Within the dead zone, no action is taken (hold current position).

### 10.5 Min Return Floor

The minimum return floor scales naturally with holding period because it is derived from the annualized risk-free rate:

```
min_return_floor = 10Y_yield * (forwardBars / 252) * 2.0  
min_short_floor  = 10Y_yield * (forwardBars / 252) * 5.0
```

With 4.5% 10Y yield:

```
forwardBars=5:  buy floor = 0.18%,  short floor = 0.45%  
forwardBars=24: buy floor = 0.86%,  short floor = 2.14%  
forwardBars=96: buy floor = 3.43%,  short floor = 8.57%
```

## 10.6 Confidence Scaling Denominator

Confidence scaling uses `min_return_floor` as its denominator, not the noise threshold. This means confidence = (pred / min\_return\_floor), which correctly measures how much excess return the model predicts above the risk-free hurdle. A prediction of 7% with a 3.43% floor gives confidence 2.0 (maximum), while a prediction of 3.5% gives confidence 1.02 (barely above neutral).

# Chapter 11

## LLM Integration

### 11.1 Two-Stage Pipeline

LLM analysis runs on a 1-hour cadence (LLM\_INTERVAL\_SEC=3600) and consists of two stages: global context generation and per-symbol analysis.

#### Stage 1: Global Context (hourly)

Fetches broad market data and asks the LLM to produce a structured market digest:

```
Data fetched:
Market indices: S&P 500, NASDAQ, VIX, DXY (dollar), Oil, Gold, 10Y yield, BTC
Each with: current price, 52-week high/low, 1w/1m/3m/1y returns
Sector ETFs (10): XLF, XLK, XLE, XLV, XLI, XLP, XLU, XLRE, XLB, XLC
Each with: same metrics as indices
Fear & Greed indices (CNN + crypto)
General news: 20 recent headlines

LLM output (structured JSON):
regime: risk-on / risk-off / rotational / crisis
themes: list of {theme, description, sector_impact_tags}
risk_factors: list of current risks with severity
opportunities: list of potential opportunities
```

#### Stage 2: Per-Symbol Analysis (hourly)

For each symbol in the candidate list (top predictions + held positions), the LLM receives the global context plus symbol-specific data and produces a structured analysis:

```
System prompt defines 5 analysis dimensions:
1. WHY: Explain the directional thesis in 1-2 sentences
2. FUNDAMENTALS: Key financial metrics and valuation
3. CATALYSTS: Upcoming events that could move the price
4. RISKS: Specific risks to the thesis
5. SYNTHESIS: Overall conviction with reasoning

Output: {s: score (0.0-1.0), m: multiplier, r: reasoning,
        bull: bull case, bear: bear case}
```

### 11.2 Score Interpretation

The LLM score is a continuous value from 0.0 to 1.0 with defined zones:

```
0.00 - 0.15: VETO (catastrophic -- do not trade under any circumstances)
0.15 - 0.35: Bearish (strong negative outlook)
0.35 - 0.48: Lean negative (mild concern)
0.48 - 0.52: Neutral (insufficient conviction either way)
0.52 - 0.65: Lean positive (mild bullish)
0.65 - 0.85: Bullish (strong positive outlook)
0.85 - 1.00: Strong conviction (very high confidence)
```

### 11.3 Time Horizon Alignment

The forward\_bars parameter is injected into the LLM prompt so the analysis matches the model's prediction horizon. A model predicting 96 hours ahead needs the LLM to evaluate 4-day catalysts, not intraday noise. The prompt explicitly states the holding period and asks the LLM to align its conviction accordingly.

## 11.4 Smart Routing

The system auto-detects free vs paid API tier from the x-ratelimit-limit-requests header on the first API call (RPM  $\leq$  15 = free tier). Cost-based routing then selects the appropriate model based on cumulative daily spend:

Daily cost thresholds:

- < \$0.10: Pro for analyst, Flash for sentiment (best quality)
- \$0.10-\$0.25: Downgrade analyst to Flash
- \$0.25-\$0.50: Further downgrade to 2.5-Flash
- > \$0.50: Lite models throughout (cheapest)
- Hard cap: \$1.00/day (~\$30/month)

Model pricing (per million tokens):

|                        |                         |
|------------------------|-------------------------|
| Gemini 3.1 Pro:        | \$2.00 in / \$12.00 out |
| Gemini 3 Flash:        | \$0.50 in / \$3.00 out  |
| Gemini 3.1 Flash Lite: | \$0.25 in / \$1.50 out  |
| Gemini 2.5 Flash:      | \$0.15 in / \$0.60 out  |

Config keys: analyst\_model\_override, sentiment\_model\_override,  
tier\_override (null = auto-detect)

# Chapter 12

## Sentiment Analysis

### 12.1 Two-Tier Architecture

Sentiment analysis uses two complementary approaches: a fast deterministic keyword scorer for live trading decisions, and an LLM-based scorer for higher accuracy when API budget allows:

- Tier 1 -- Keyword scoring: Instant, no API call, deterministic. Used for all live trading decisions where latency matters.
- Tier 2 -- LLM batch scoring: Higher accuracy, tiered by article recency. Runs in background, results merged into daily aggregates.

### 12.2 Keyword Scoring Algorithm

The keyword engine uses a comprehensive dictionary of financial terms with negation-aware scoring:

```
Dictionary size:
  70+ positive words, 100+ negative words
  20+ positive phrases, 25+ negative phrases

Phase 1: Phrase matching (word-boundary regex, negation-aware)
  Positive phrases: 'all-time high' (1.5), 'beat expectations' (1.5), ...
  Negative phrases: 'death cross' (-1.5), 'slashed price target' (-2.0), ...
  If negator within 3-word window: flip polarity * 0.7

Phase 2: Single-word scoring with bidirectional negation check
  Each positive keyword: +1.0 (or -0.7 if negated)
  Each negative keyword: -1.0 (or +0.7 if negated)
  Negators: 'not', 'no', 'never', 'neither', 'barely', 'hardly', etc.

Phase 3: Tanh normalization
  scale = 0.4 / sqrt(word_count / 10)
  score = tanh(raw * scale) -> smooth to (-1.0, +1.0)

Combination: headline_score * 0.6 + summary_score * 0.4
```

### 12.3 LLM Tiering by Article Recency

When LLM scoring is available, articles are prioritized by recency to allocate the best (most expensive) models to the most important content:

```
Newest 20% of articles -> Pro model (highest accuracy)
Next 40% of articles   -> Flash model (good accuracy, lower cost)
Remaining 40%         -> Lite model (adequate accuracy, cheapest)

Rationale: Recent news has the most impact on near-term price action.
Older articles are less actionable and don't justify premium model costs.
```

### 12.4 Sentiment Gate Integration

The sentiment gate applies a multiplicative factor to position sizing based on symbol-specific news sentiment:

```
Sentiment multiplier mapping:
  score <= -0.5: mult = 0.15 (catastrophic news: hack, fraud)
  score <= -0.3: mult = 0.35 (heavy bearish news)
  score <= -0.1: mult = 0.70 (mild caution)
  score >= 0.4:  mult = 1.35 (strong positive news)
  score >= 0.2:  mult = 1.20 (positive news)
  else:         mult = 1.00 (neutral)
```

Design principle: the ML model already sees Daily\_Sentiment as a training feature, so broad market sentiment adjustments would double-count. Symbol-specific breaking news IS new information the model cannot see, so hard gates for catastrophic events (hack, fraud, bankruptcy) are applied as a safety net.

## 12.5 Fear & Greed Index

Crypto-specific bubble protection: when the Fear & Greed Index reaches extreme greed ( $\geq 90$ ), a 0.7x sizing reduction is applied. This is the only Fear & Greed integration point, because the ML model already captures sentiment via the Daily\_Sentiment feature:

Crypto: alternative.me API (free, no auth required)

|                    |                                                 |
|--------------------|-------------------------------------------------|
| 0-24: Extreme Fear | 25-49: Fear                                     |
| 50: Neutral        | 51-74: Greed                                    |
| 75-89: Greed       | 90-100: Extreme Greed -> 0.7x bubble protection |

Stocks: CNN Fear & Greed (includes VIX component, AAI survey, etc.)

Cache: 5 minutes

# Chapter 13

## GARCH Volatility Forecasting

Based on Robert Engle's ARCH/GARCH framework (Nobel Prize 2003). GARCH captures volatility clustering -- the empirical observation that high-volatility periods tend to persist, and low-volatility periods tend to persist. This provides a forward-looking volatility estimate that is superior to backward-looking measures like ATR or simple standard deviation.

### 13.1 GARCH(1,1) Model

$$\sigma_t^2 = \omega + \alpha * r_{t-1}^2 + \beta * \sigma_{t-1}^2$$

Where:

$\sigma_t^2$ : conditional variance at time t  
 $\omega$ : baseline variance (long-run average volatility)  
 $\alpha$ : reaction coefficient (how fast vol responds to shocks)  
 $\beta$ : persistence coefficient (how long high/low vol persists)  
 $r_{t-1}$ : previous period return

Typical values:  $\alpha \sim 0.05-0.15$ ,  $\beta \sim 0.80-0.95$   
 $\alpha + \beta < 1.0$  required for stationarity

### 13.2 EGARCH Preference

The system tries EGARCH (Exponential GARCH) first, falling back to standard GARCH(1,1) if EGARCH fitting fails. EGARCH captures the asymmetric volatility response -- the empirical fact that market crashes increase volatility more than rallies of equivalent magnitude. This asymmetry is especially important for stop-loss calibration, as downside volatility is what triggers stops.

### 13.3 Implementation Details

Implementation (volatility.py):

1. Try EGARCH first (captures crash asymmetry)
2. Fall back to standard GARCH(1,1) if EGARCH fails
3. Require  $\geq 100$  data points to fit (statistical reliability)
4. Cache fitted model for 1 hour (vol is persistent, refit expensive)
5. Forecast 1-step-ahead variance -> sqrt -> sigma

Used for:

Position sizing:  $\text{ratio} = \text{target\_vol}(2\%) / \text{sigma}$ , clamped [0.5, 2.0]  
 Effect: high-vol asset ( $\text{sigma}=4\%$ ) -> 0.5x size (half position)  
           low-vol asset ( $\text{sigma}=1\%$ ) -> 2.0x size (double position)  
 -> Equalizes dollar risk per position across the portfolio

# Chapter 14

## Macro Regime Detection

The macro regime system combines multiple independent indicators to classify the overall market environment. Each indicator has specific thresholds and actions:

### 14.1 VIX (CBOE Volatility Index)

Source: yfinance (^VIX), 1-hour cache

Thresholds:

|            |           |                                                                                         |
|------------|-----------|-----------------------------------------------------------------------------------------|
| VIX < 15:  | Normal    | -> sizing 1.0x, no restrictions                                                         |
| VIX 15-25: | Caution   | -> sizing 0.8x                                                                          |
| VIX 25-35: | Defensive | -> sizing 0.5x, block risky stock entries<br>(only safe-havens: TLT, GLD, etc. allowed) |
| VIX > 35:  | Crisis    | -> sizing 0.3x, HALT ALL stock entries                                                  |

The VIX is the primary macro signal because it reflects real-time options pricing and represents aggregate market fear.

### 14.2 STLFSI2 (Financial Stress Index)

Source: FRED CSV download, 1-day cache

Published by: St. Louis Federal Reserve (weekly)

Components: 18 financial indicators including yield spreads, volatility measures, and funding stress indicators

Thresholds:

|                 |             |                                          |
|-----------------|-------------|------------------------------------------|
| STLFSI2 <= 1.0: | Normal      | -> no adjustment                         |
| STLFSI2 > 1.0:  | High stress | -> sizing *= 0.5, stops *= 0.8 (tighter) |

Captures systemic financial stress that VIX alone may miss (e.g., credit market freeze, interbank lending stress).

### 14.3 CAPE Ratio (Cyclically Adjusted PE)

Estimation: SPY trailing PE \* 1.6 (approximation), 1-day cache

z-score = (CAPE - 25.0) / 8.0

Thresholds:

|                 |            |                        |
|-----------------|------------|------------------------|
| z-score <= 1.5: | Normal     | -> no adjustment       |
| z-score > 1.5:  | Overvalued | -> stock sizing *= 0.7 |

Based on Shiller's CAPE (Nobel 2013). Historically, CAPE > 35 signals overvaluation. The z-score normalization allows the system to adapt to slowly rising market valuations.

### 14.4 Stablecoin Peg Monitoring

Source: Alpaca quotes (USDT, USDC), 5-minute cache

Measures: deviation from \$1.00 peg

Thresholds:

|               |           |                                                                                                     |
|---------------|-----------|-----------------------------------------------------------------------------------------------------|
| depeg > 2.0%: | EMERGENCY | -> halt ALL crypto trading, flatten positions<br>Signals potential exchange insolvency or contagion |
| depeg > 0.5%: | WARNING   | -> tighten crypto stops *= 0.7 (30% tighter)<br>Signals stress in crypto infrastructure             |

Historical precedent: UST depeg in May 2022 preceded \$40B collapse. Early detection of stablecoin stress can prevent catastrophic losses.

## 14.5 Regime Output

MacroRegime dataclass output:

```
sizing_mult:    0.0 to 1.0+ (multiplicative position scaling)
stop_mult:      0.7 to 1.0 (tighten stops when stressed)
regime_label:   'normal', 'caution', 'defensive', 'crisis'
stablecoin_alert: True/False (crypto-specific warning)
```

Properties:

```
should_halt_stocks:    VIX > 35
should_block_risky_entries: VIX > 25
```

# Chapter 15

## Correlation-Aware Sizing

Based on Markowitz portfolio theory (Nobel 1990). The correlation system prevents concentrated portfolios where multiple positions move together, which amplifies drawdowns during market stress.

### 15.1 Correlation Matrix Estimation

Process:

1. Fetch 30-bar rolling returns for all symbols in universe
2. Compute correlation matrix:
  - Try Ledoit-Wolf shrinkage estimator (sklearn)  
Regularizes toward structured estimator, robust to small samples
  - Fallback to numpy corrcoef if sklearn unavailable
3. Cache result for 1 hour (recalculated every 10 cycles = ~5 min)

### 15.2 Entry Blocking

Before any buy, the system computes the average absolute correlation between the candidate symbol and all currently held positions:

```
avg_corr = mean(abs(corr(candidate, each_held_position)))

If avg_corr > 0.7: REJECT entry entirely
-> Symbol is too correlated with existing portfolio
-> Adding it would increase concentration risk without diversification

Example: Holding AAPL, MSFT, GOOGL. Candidate: META
corr(META, AAPL) = 0.82, corr(META, MSFT) = 0.78, corr(META, GOOGL) = 0.75
avg_corr = 0.783 > 0.7 -> REJECTED (tech concentration)
```

### 15.3 Sizing Reduction

For candidates that pass the 0.7 threshold, a sizing reduction is applied proportional to the correlation level:

```
sizing_factor = max(0.5, 1.0 - 0.5 * avg_corr)

Examples:
avg_corr = 0.0 -> factor = 1.0x (fully uncorrelated, no reduction)
avg_corr = 0.2 -> factor = 0.9x (slight reduction)
avg_corr = 0.4 -> factor = 0.8x (moderate reduction)
avg_corr = 0.6 -> factor = 0.7x (significant reduction)
avg_corr = 0.69 -> factor = 0.655x (near rejection threshold)

Floor of 0.5 prevents complete elimination -- even highly correlated
assets may have valid idiosyncratic signals.
```

### 15.4 Ledoit-Wolf Shrinkage

The Ledoit-Wolf shrinkage estimator is critical for correlation estimation with small sample sizes (30 bars). Classical sample correlation matrices are notoriously unstable with fewer observations than dimensions. Ledoit-Wolf regularizes the sample covariance matrix toward a structured target (scaled identity), producing more stable and reliable correlation estimates. This addresses the well-known limitation Markowitz himself acknowledged: portfolio optimization is 'extremely sensitive to estimation errors in the input parameters.'

# Chapter 16

## HMM Regime Detection

A 3-state Gaussian Hidden Markov Model classifies the market into regimes based on return distributions. The key insight from Sargent and Sims (Nobel 2011): markets alternate between distinct states (trending, mean-reverting, volatile), and the current state is only probabilistically observable from price data.

### 16.1 Model Specification

```
Model: GaussianHMM (from hmmlearn library)
  n_states = 3
  covariance_type = 'full'
  Minimum data: 200 bars (statistical reliability)
  Input: 1D array of log returns

Fitting: Baum-Welch algorithm (Expectation-Maximization)
  Iteratively estimates: transition matrix, emission means, emission covariances
  Refit daily (HMM fitting is expensive, regimes change slowly)
  Cached for 24 hours between refits
```

### 16.2 State Classification

After fitting, the three states are sorted by their mean return to assign semantic labels:

```
State sorting by emission mean:
  Lowest mean return -> Bear state
  Middle mean return -> Neutral state
  Highest mean return -> Bull state

Additional classification:
  If neutral state vol > 1.5x median vol of all states:
    -> Reclassified as High-Volatility state
```

### 16.3 Trading Parameter Adjustments

```
Bull regime:
  sizing = 1.2x (lean in, momentum likely to continue)
  threshold = 0.8x (lower bar for entry, more permissive)
  stops = 1.0x (normal)

Neutral regime:
  sizing = 1.0x (no change)
  threshold = 1.0x (no change)
  stops = 1.0x (normal)

Bear regime:
  sizing = 0.3x (70% reduction, very defensive)
  threshold = 1.5x (50% higher bar for entry)
  stops = 0.8x (20% tighter, protect capital)

High-volatility state:
  sizing = 0.5x (50% reduction)
  threshold = 1.2x (higher bar)
  stops = 1.3x (30% wider to avoid whipsaw)
```

### 16.4 Whipsaw Prevention (3-Bar Persistence)

To prevent rapid oscillation between regimes on minor price fluctuations, the system requires 3 consecutive bars in a new regime before switching:

```
_smooth_regime(current_state, state_history):  
    If last 3 states are all the same new state:  
        -> Switch to new regime (confirmed transition)  
    Else:  
        -> Stay in previous regime (insufficient confirmation)
```

This prevents:

```
Cycle 1: Bull -> (noise) -> Bear -> (noise) -> Bull  
From causing rapid sizing changes (1.2x -> 0.3x -> 1.2x)  
Instead: stays in previous regime until new one persists
```

# Chapter 17

## Data Pipeline

### 17.1 Storage Format

All market data is stored in Parquet format with Snappy compression, with CSV files maintained for backward compatibility. Parquet provides significant advantages for time series data: columnar storage reduces I/O for feature computation, Snappy compression reduces disk usage by 3-5x, and read performance is 5-10x faster than CSV.

### 17.2 Three-Source Fallback

Data harvesting uses a cascading fallback strategy to maximize data availability:

1. Alpaca (primary): Hourly bars  
Crypto: 2021+, Stock: 2016+  
Chunked 6-month fetches with exponential backoff (4, 8, 16, 32s)  
Rate limit: adaptive pacing (up to 30s between chunks on 429)
2. yfinance (secondary): Last 730 days hourly  
Pre/post market data included  
Flattens MultiIndex columns for yfinance 0.2.x compatibility
3. CryptoCompare (crypto only, tertiary): Free API, no key required  
2000-bar backward chunks  
Filters bars with close > 0 (removes invalid data)

### 17.3 Incremental Harvesting

After initial data download, subsequent harvests only fetch bars newer than the last run, with a 48-hour overlap for safety (catches any bars that may have been delayed or corrected by the exchange):

- Process:
1. Read existing Parquet file, find most recent timestamp per ticker
  2. Fetch new bars from (last\_timestamp - 48 hours) to now
  3. Merge new bars with existing data, deduplicate on timestamp
  4. Write atomically (temp file then os.replace, crash-safe)

Row capping: If total rows > max\_rows, keep most recent per ticker  
(balanced across symbols to prevent one ticker from dominating)

### 17.4 Multi-Horizon Targets

Target variables are computed for multiple prediction horizons simultaneously:

$$\text{Target\_Return\_N} = (\text{close}[t+N] - \text{close}[t]) / \text{close}[t] * 100$$

N in [12, 18, 24, 32, 48] bars ahead (configurable, expandable to 96)  
Optuna searches across horizons to find the most predictable timeframe.  
Stale targets (e.g., Target\_Return\_6 from old configs) are auto-removed.

### 17.5 Data Validation

- Gap detection: identifies missing bars in the time series
- NaN/Inf checks: removes rows with invalid numeric values
- Per-ticker coverage: ensures minimum bar count per symbol
- Atomic writes: write-then-rename pattern prevents partial file corruption on crash or power loss (important for Jetson)

which may lose power)

# Chapter 18

## Research Foundations

---

The system's design decisions are grounded in peer-reviewed research and Nobel Prize-winning economic theory. Below is a mapping of key academic contributions to specific implementation decisions in the trading system.

### 18.1 Harry Markowitz (1952, Nobel 1990) -- Modern Portfolio Theory

PAPER: 'Portfolio Selection', Journal of Finance, 1952. Markowitz demonstrated that portfolio risk depends not just on individual asset risk but on the correlations between assets. Diversification reduces total portfolio risk without proportionally reducing expected return.

IMPLEMENTATION: portfolio.py uses correlation-aware position sizing (Chapter 15). Entry is blocked when avg correlation > 0.7. Sizing is reduced proportionally to correlation. The Sharpe ratio (derived from Markowitz's framework by his student William Sharpe) is the optimization objective for the Optuna search.

### 18.2 Robert Engle (Nobel 2003) -- ARCH/GARCH

DISCOVERY: Autoregressive Conditional Heteroskedasticity -- volatility itself is predictable. High-volatility periods cluster together, as do low-volatility periods. This is the most important insight for risk management: we can forecast tomorrow's volatility from today's.

IMPLEMENTATION: volatility.py implements GARCH(1,1) and EGARCH for volatility forecasting (Chapter 13). Used for position sizing (target 2% daily vol per position), stop-loss calibration, and risk parity across the portfolio.

### 18.3 Daniel Kahneman (Nobel 2002) -- Prospect Theory

DISCOVERY: People feel losses approximately 2x as painfully as equivalent gains (loss aversion). This creates systematic behavioral biases: panic selling at bottoms (our bot buys), holding losers too long hoping for recovery (our mechanical stops cut losses), and selling winners too early to lock in gains (our trailing stops let winners run).

IMPLEMENTATION: The entire mechanical execution system is the implementation. By removing human emotional decision-making, the bot exploits the behavioral biases of other market participants. The hard-stop lockout (Chapter 9) specifically prevents revenge trading -- a manifestation of loss aversion.

### 18.4 Richard Thaler (Nobel 2017) -- Behavioral Economics

DISCOVERY: Built on Kahneman's work to show that behavioral biases persist even among sophisticated investors. Calendar effects (Turn of Month, January effect) exist because institutional cash flows are behaviorally driven.

IMPLEMENTATION: Calendar features (Hour/Day/Month sin/cos, Turn\_of\_Month flag) in Chapter 3. Sentiment gate design recognizes that news-driven panic creates predictable overreaction patterns.

### 18.5 J.L. Kelly Jr. (1956) -- Kelly Criterion

PAPER: 'A New Interpretation of Information Rate', Bell System Technical Journal. Kelly proved that betting a fraction  $f^*$  = edge/odds maximizes the long-term geometric growth rate of capital. Half-Kelly ( $f^*/2$ ) sacrifices only 25% of growth rate while halving variance and maximum drawdown.

IMPLEMENTATION: trading\_utils.py computes half-Kelly from the last 200 trades, clamped to [0.05, 0.25]. Minimum 50 trades required before Kelly activates. VIX-scaled Kelly further adjusts based on market conditions (Step 2 of the risk stack).

## 18.6 Baum-Welch Algorithm -- HMM Training

The Baum-Welch algorithm (a special case of Expectation-Maximization) fits the Hidden Markov Model parameters from observed returns. It iteratively estimates the transition probabilities between states and the emission distributions within each state, converging to a local maximum of the likelihood function.

IMPLEMENTATION: `regime_detector.py` fits a 3-state Gaussian HMM using `hmmlearn`'s implementation of Baum-Welch (Chapter 16). Daily refit, 200+ bar minimum, 3-bar persistence hysteresis for whipsaw prevention.

## 18.7 Hochreiter & Schmidhuber (1997) -- LSTM

PAPER: 'Long Short-Term Memory', Neural Computation. LSTMs solved the vanishing gradient problem that prevented standard RNNs from learning long-range dependencies. The gated architecture (input, forget, output gates) allows the network to selectively remember or forget information over hundreds of timesteps.

IMPLEMENTATION: `model_v2.py` uses LSTM as the primary temporal encoder (Chapter 2). The LSTM captures sequential patterns in hourly price data -- momentum, mean reversion, volatility clustering -- that are invisible to static feature-based models.

## 18.8 Vaswani et al. (2017) -- Attention Mechanism

PAPER: 'Attention Is All You Need', NeurIPS 2017. Multi-head self-attention allows the model to learn which parts of the input sequence are most relevant to the output, with different attention heads learning different aspects of relevance.

IMPLEMENTATION: `model_v2.py` adds `MultiHeadAttention` on top of the LSTM output (Chapter 2). Self-attention with  $Q=K=V=\text{Istm\_output}$  allows the model to learn which timesteps in the lookback window are most informative for predicting forward returns. The residual connection and `LayerNorm` ensure the attention layer enhances rather than degrades the LSTM's temporal signal.